1 Introdução

Nesta questão, implementei diferentes métodos de medição de tempo do produto de matrizes por meio de listas de listas, em `NumPy` e, naturalmente, com o operador nativo `@`. O objetivo é analisar o desempenho de cada abordagem e como ele varia com o tamanho das matrizes. Ao compilar os arquivos, é possível ver as tabelas de comparação de cada medição de tempo com as funções mensuradas.

2 Medidores de Tempo

Foram utilizados quatro métodos de medição de desempenho:

- `perf_counter`: Alta resolução; usado com uma frequência maior;
- `time.time`: Menor resolução; sensível a ajustes do sistema operacional.
- `process_time`: Mede apenas o tempo de CPU do processo; ignora I/O e pausas.
- `timeit`: Desabilita o *garbage collector* durante a medição; reduz ruído.

2.1 Decisões de projeto para evitar distorções

- **Mesma matriz para todos os medidores.** A e B são geradas uma só vez por configuração e reutilizadas em todos os testes, o que garante dados numéricos idênticos.
- **Aquecimento (*warm-up*).** Cada função é executada uma vez antes das medições, forçando o carregamento do bytecode e o mapeamento de memória pelo SO.
- **Mínimo de 10 repetições.** Registra-se a *média* das 10 repetições;
- **Alocação de C dentro da função.** Custo contado no algoritmo; subtrair ele separadamente iria acrescentar um segundo erro de medição.

3 Resultados em Python

Abaixo estão as minhas conclusões tiradas a partir das tabelas geradas pelos dados colhidos dos arquivos `ex1.py` e `ex1.jl`:

Tabela 1: Comparação entre todas as implementações — `perf_counter` (s) e razão vs. `mat_prod`

Dimensões	mat_prod(s)	prod_np(s)	R_{np}	dot (s)	R_{dot}	@ nativo (s)	$R_{@}$
5×5	0.000073	0.000313	4.3×	0.000138	1.9×	0.000012	0.16×
10×10	0.000385	0.002454	6.4×	0.000563	1.5×	0.000020	0.05×
10×50	0.001382	0.012467	9.0×	0.000638	0.46×	0.000018	0.01×
20×100	0.008844	0.091344	10.3×	0.001594	0.18×	0.000020	<0.01×
50×10	0.005839	0.042314	7.2×	0.011149	1.9×	0.000020	<0.01×
100×20	0.044311	0.361544	8.2×	0.043944	0.99×	0.000080	<0.01×
50×50	0.025702	0.217838	8.5×	0.009371	0.36×	0.000036	<0.01×
100×100	0.183489	1.775421	9.7×	0.044401	0.24×	0.000197	<0.01×
150×150	0.524531	4.515325	8.6×	0.071192	0.14×	0.000208	<0.01×
200×200	2.014267	9.157846	4.5×	0.118689	0.06×	0.000383	<0.01×
300×300	3.879391	29.850443	7.7×	0.293546	0.08×	0.001269	<0.01×

R_x = razão tempo_x / $\text{tempo}_{\text{lista}}$. Razão < 1: função mais rápida que `mat_prod`.

Na tabela acima, as funços `prod_np` e `dot` se referem às funções `mat_prod_np` e `mat_prod_dot` do arquivo ex1.py.

3.1 Discussão

Os quatro métodos de medição apresentam resultados consistentes para matrizes maiores (a partir de 100×100), pois o tempo de execução é suficiente para diluir erros de resolução e interferências do SO; já pra matrizes pequenas, tem maior variabilidade, especialmente em `time.time` e `process_time`, por causa da baixa precisão em escalas de microssegundos, enquanto o `timeit` tende a registrar menores tempos por desativar o *garbage collector* e se aproximar do custo puro da execução.

Os resultados mostram que o desempenho depende fortemente de onde o laço de computação é executado. Quando usamos um triplo laço em Python sobre um `numpy.ndarray`, cada acesso envolve overhead do interpretador (verificação de tipo, indexação e conversão), o que torna `mat_prod_np` significativamente mais lenta do que a versão com listas. Isso ocorre porque, nesse regime escalar, o NumPy não consegue explorar as operações vetorizadas em código compilado, que são sua maior vantagem.

O custo computacional cresce como $\mathcal{O}(mnp)$, o que é confirmado empiricamente: dobrar as dimensões resulta em um aumento de aproximadamente $8\times$ no tempo de execução. Além disso, testes com diferentes formatos de matrizes, mas mesmo número total de operações, apresentam tempos semelhantes, o que indica que o volume total de operações domina o desempenho, enquanto a forma da matriz tem um impacto secundário.

Finalmente, o operador nativo `@` apresenta o melhor desempenho em todos os cenários. Como resultado da sua otimização, ele é melhor em várias ordens de magnitude ao ser comparado com implementações com laços explícitos (as que fizemos).

4 Resultados em Julia

Tabela 2: Comparação entre implementações em Julia — `perf_counter` (s) e razão vs. `mat_prod`

Dimensões	**lista (s)**	**array loop (s)**	$\mathbf{R_{array}}$	**dot (s)**	$\mathbf{R_{dot}}$	*** nativo (s)**	$\mathbf{R_{*}}$
5×5	0.000002	0.000001	0.68×	0.000007	3.15×	0.000001	0.69×
10×10	0.000007	0.000005	0.74×	0.000025	3.63×	0.000001	0.20×
10×50	0.000042	0.000038	0.90×	0.000064	1.52×	0.000006	0.15×
20×100	0.000289	0.000357	1.24×	0.000431	1.49×	0.000015	0.05×
50×10	0.000230	0.000190	0.83×	0.000767	3.34×	0.000013	0.06×
100×20	0.001674	0.002738	1.64×	0.007553	4.51×	0.000161	0.10×
50×50	0.000950	0.001050	1.11×	0.026553	27.95×	0.000065	0.07×
100×100	0.007187	0.008446	1.18×	0.012529	1.74×	0.000204	0.03×
150×150	0.023526	0.030347	1.29×	0.069626	2.96×	0.000543	0.02×
200×200	0.055890	0.070614	1.26×	0.107536	1.92×	0.000701	0.01×
300×300	0.167537	0.241916	1.44×	0.630023	3.76×	0.003001	0.02×
500×300	0.698828	0.681923	0.98×	1.666049	2.38×	0.016730	0.02×
500×500	1.096132	1.138747	1.04×	3.936143	3.59×	0.028377	0.03×

$\mathrm{R_x}$ = razão $\mathrm{tempo_x}$ / $\mathrm{tempo_{lista}}$. Razão < 1: função mais rápida que `mat_prod`.

4.1 Discussão

Os resultados mostram três contrastes bem claros de desempenho. Pra matrizes pequenas, todas as abordagens têm tempos muito próximos, com variações pequenas dominadas por *overhead* e precisão de medição; ainda assim, o operador nativo `@` já aparece como o mais rápido. À medida que o tamanho cresce, o comportamento cúbico $\mathcal{O}(n^3)$ fica mais evidente nas implementações com laços (`mat_prod` e `mat_prod_np`), que crescem rapidamente e mantêm desempenho semelhante, com uma leve desvantagem pra o NumPy quando usado sem a sua vetorização.

A função `mat_prod_dot` apresenta desempenho inconsistente: em matrizes pequenas e médias, ela frequentemente é mais lenta por causa do custo de múltiplas chamadas ao `mat_prod_dot`, só começando a se beneficiar parcialmente em dimensões maiores; mas ainda assim sem competir com a abordagem totalmente vetorizada.

Finalmente, o operador nativo `@` é mais vantajoso na questão de tempo em todos os cenários relevantes, com ganhos que chegam a dezenas ou até centenas de vezes para matrizes grandes.

5 Conclusão

Os resultados que obtemos refletem diretamente as diferenças entre Python e Julia. Enquanto em Python o uso de laços explícitos introduz overhead do interpretador, tornando as implementações mais lentas, Julia executa esses mesmos laços de forma eficiente por compilação JIT, gerando um código otimizado próximo de C. Isso explica por que, mesmo sem depender de bibliotecas externas, Julia apresenta melhor desempenho. Já em Python, ganhos significativos só aparecem ao utilizar operações vetorizadas (como `@`), que transferem o trabalho para rotinas compiladas. Assim, temos que não apenas os resultados obtidos condizem com o esperado, como eles evidenciam muito bem esse contraste entre as duas linguagens.